```
// ... existing code ... mapping(address => uint256) public balances; mapping(address =>
uint256) public lockedUntil;
// --- Shift Node & Streaming Creator Vault Accounting --- mapping(address => uint256) public
creatorVaultedBalances; mapping(address => uint256) public creatorAgreedUnlockTime;
mapping(address => bool) public creatorDateApproved; mapping(address => uint256) public
viewerClaimableRewards;
bool public paused;
// --- Events --- event Deposited(address indexed account, uint256 amount, uint256
lockDuration); event Withdrawn(address indexed account, uint256 amount); event
GovernanceUpdated(address indexed newSafe); event EmergencyPaused(bool status); event
ShiftNodeRevenueDistributed( address indexed creator, uint256 totalAmount, uint256
creatorImmediate, uint256 creatorVaulted, uint256 platformShare, uint256 unlockTimestamp );
event StreamingRevenueDistributed( address indexed creator, address indexed viewerPool,
uint256 totalAmount, uint256 creatorImmediate, uint256 creatorVaulted, uint256 viewerShare,
uint256 platformShare, uint256 unlockTimestamp ); event CreatorUnlockDateApproved(address
indexed creator, bool approved); event CreatorVaultWithdrawn(address indexed creator, uint256
amount); event ViewerRewardClaimed(address indexed viewer, uint256 amount);
// --- Modifiers --- modifier onlyOwner() { require(msg.sender == owner, "KEMVault:
UNAUTHORIZED"); _; }
modifier onlyAuthorizedGovernance() { require(msg.sender == owner || msg.sender ==
governanceSafe, "KEMVault: NOT_GOVERNANCE"); _; }
modifier whenNotPaused() { // ... existing code ... emit EmergencyPaused(_paused); }
/ * @notice Distribute Shift Node revenue: * - 75% Creator Total (25% immediate payout + 50%
vaulted until approved date or agreed time) * - 25% Platform (routed to Governance Safe) */
function distributeShiftNodeRevenue(address creator, uint256 agreedUnlockTimestamp)
external payable whenNotPaused { require(msg.value > 0, "KEMVault: ZERO_REVENUE");
require(creator != address(0), "KEMVault: ZERO_CREATOR");
uint256 creatorImmediate = (msg.value * 2500) / 10000; // 25% immediately available uint256
creatorVaulted = (msg.value * 5000) / 10000; // 50% vaulted (total 75% creator) uint256
platformShare = msg.value - creatorImmediate - creatorVaulted; // 25% platform
creatorVaultedBalances[creator] += creatorVaulted; if (agreedUnlockTimestamp >
creatorAgreedUnlockTime[creator]) { creatorAgreedUnlockTime[creator] =
agreedUnlockTimestamp; }
// 25% immediate creator transfer (bool sentCreator, ) = payable(creator).call{value:
creatorImmediate}(""); require(sentCreator, "KEMVault: CREATOR_TRANSFER_FAILED");
// 25% platform transfer to Governance Safe (bool sentPlatform, ) =
payable(governanceSafe).call{value: platformShare}(""); require(sentPlatform, "KEMVault:
PLATFORM_TRANSFER_FAILED");
emit ShiftNodeRevenueDistributed( creator, msg.value, creatorImmediate, creatorVaulted,
platformShare, creatorAgreedUnlockTime[creator] ); }
/ * @notice Distribute Streaming revenue: * - 75% Creator Total (25% immediate payout + 50%
vaulted until approved date or agreed time) * - 10% Viewers (allocated to pull-based viewer
reward pool) * - 15% Platform (routed to Governance Safe) */ function
distributeStreamingRevenue( address creator, address viewerPoolRecipient, uint256
agreedUnlockTimestamp ) external payable whenNotPaused { require(msg.value > 0,
"KEMVault: ZERO_REVENUE"); require(creator != address(0), "KEMVault: ZERO_CREATOR");
require(viewerPoolRecipient != address(0), "KEMVault: ZERO_VIEWER_RECIPIENT");
uint256 creatorImmediate = (msg.value * 2500) / 10000; // 25% immediately available uint256
```

```
creatorVaulted = (msg.value * 5000) / 10000; // 50% vaulted (total 75% creator) uint256
viewerShare = (msg.value * 1000) / 10000; // 10% viewers uint256 platformShare = msg.value -
creatorImmediate - creatorVaulted - viewerShare; // 15% platform
creatorVaultedBalances[creator] += creatorVaulted; if (agreedUnlockTimestamp >
creatorAgreedUnlockTime[creator]) { creatorAgreedUnlockTime[creator] =
agreedUnlockTimestamp; }
viewerClaimableRewards[viewerPoolRecipient] += viewerShare;
// 25% immediate creator transfer (bool sentCreator, ) = payable(creator).call{value:
creatorImmediate}(""); require(sentCreator, "KEMVault: CREATOR_TRANSFER_FAILED");
// 15% platform transfer to Governance Safe (bool sentPlatform, ) =
payable(governanceSafe).call{value: platformShare}(""); require(sentPlatform, "KEMVault:
PLATFORM_TRANSFER_FAILED");
emit StreamingRevenueDistributed( creator, viewerPoolRecipient, msg.value, creatorImmediate,
creatorVaulted, viewerShare, platformShare, creatorAgreedUnlockTime[creator] ); }
/ * @notice Governance or Owner approves early or scheduled release date for a creator's 50%
vaulted share. */ function setCreatorUnlockApproval(address creator, bool approved) external
onlyAuthorizedGovernance { creatorDateApproved[creator] = approved; emit
CreatorUnlockDateApproved(creator, approved); }
/ * @notice Withdraw the 50% vaulted creator allocation once the agreed time is reached OR
date is approved. */ function withdrawCreatorVault(uint256 amount) external whenNotPaused {
require(creatorVaultedBalances[msg.sender] >= amount, "KEMVault:
INSUFFICIENT_VAULT_BALANCE"); require( creatorDateApproved[msg.sender] ||
(creatorAgreedUnlockTime[msg.sender] > 0 && block.timestamp >=
creatorAgreedUnlockTime[msg.sender]), "KEMVault:
VAULT_LOCKED_UNTIL_APPROVED_OR_AGREED_TIME" );
creatorVaultedBalances[msg.sender] -= amount;
(bool success, ) = payable(msg.sender).call{value: amount}(""); require(success, "KEMVault:
VAULT_TRANSFER_FAILED");
emit CreatorVaultWithdrawn(msg.sender, amount); }
/ * @notice Viewers claim their 10% streaming reward share (pull-based accounting). */ function
claimViewerReward() external whenNotPaused { uint256 reward =
viewerClaimableRewards[msg.sender]; require(reward > 0, "KEMVault:
NO_VIEWER_REWARDS");
viewerClaimableRewards[msg.sender] = 0;
(bool success, ) = payable(msg.sender).call{value: reward}(""); require(success, "KEMVault:
VIEWER_CLAIM_FAILED");
emit ViewerRewardClaimed(msg.sender, reward); }
// Fallback receiver receive() external payable { // ... existing code ...
```